Our notebook begins with some instructions for setting up your secret for this project. First, we connect our Vector Database and LLM using the LangChain framework. We are using the integration from Databricks with LangChain to create a function called `get_retriever`, which calls the VS index that you created in *Chapter 4*. We include the host and access token in the function because our goal is for this chatbot to connect it to a hosted application we deploy in later chapters. The application will require the host and access token to be able to authenticate. *Figure 6.19* shows the code to call the previously created index, create the retriever, and run a quick test see the top relevant document returned for a user's question.

```
from databricks.vector_search.client import VectorSearchClient
from langchain.vectorstores import DatabricksVectorSearch
from langchain.embeddings import DatabricksEmbeddings

# NOTE: your question embedding model must match the one used in the chunk in the previous model
embedding_model = DatabricksEmbeddings(endpoint="databricks-bge-large-en")
vsc_endpoint_name = "ml_action_vs"
index_name = f"{catalog}.{database_name}.docs_vsc_idx_cont"

def get_retriever(persist_dir: str = None):
    os.environ["DATABRICKS_HOST"] = host
    #Get the vector search index
    vsc = VectorSearchClient(workspace_url=host, personal_access_token=os.environ["DATABRICKS_TOKEN"])
    print("\n")
    vs_index = vsc.get_index(
        endpoint_name=vsc_endpoint_name,
        index_name=index_name)
    # Create the retriever
    vectorstore = DatabricksVectorSearch(
        vs_index, text_column="content", embedding=embedding_model)
    return vectorstore.as_retriever()

# test our retriever
vectorstore = get_retriever()
similar_documents = vectorstore.get_relevant_documents("What is GPT? ")
print(f"Relevant documents: {similar_documents[0]}")
```

Figure 6.19 – Testing the retrieval component with the LangChain

Here is the first element that was retrieved from the Databricks VS for the question, "What is GPT?"

```
Relevant documents: page_content='7.1 LLM Conclusion (GPT-4's Version)\nGenerative Pre-trained Transformers (GPTs) generate profound transformations, garnering poten
tial technolog- ical growth, permeating tasks, greatly impacting professions. This study probes GPTs' potential trajectories, presenting a groundbreaking rubric to g
auge tasks' GPT exposure, particularly in the U.S. labor market.\n7.2 LLM Conclusion (Author-Augmented Version)\nGenerative Pre-trained Transformers (GPTs) generate
profound transformations, garnering potential techno- logical growth, permeating tasks, gutting professional management. Gauging possible trajectories? Generate pion
eering taxonomies, gather policymakers together, generalize past today.\nAcknowledgments\nThank you to the group of annotators who helped us annotate task exposure,
including Muhammad Ahmed Saeed, Bongane Zitha, Merve Özen Şenen, J.J., and Peter Hoeschele. We also thank Lauryn Fuld, Ashley Glat, Michael Lampe, and Julia Susser f
or excellent research assistance. We thank Miles Brundage for significant feedback on this paper.\nWe thank Todor Markov and Vik Goel for setting up the infrastructu
re we use to run our rubrics with GPT-4. We thank Lama Ahmad, Donald Bakong, Seth Benzell, Erik Brynjolfsson, Parfait Eloundou-Enyegue, Carl Frey, Sarah Giroux, Gill
ian Hadfield, Johannes Heidecke, Alan Hickey, Eric Horvitz, Shengli Hu, Ashyana Kachra, Christina Kim, Katya Klinova, Daniel Kokotajlo, Gretchen Krueger, Michael Lam
pe, Aalok Mehta, Larissa Schiavo, Daniel Selsam, Sarah Shoker, Prasanna Tambe, and Jeff Wu for feedback and edits at various stages of the project.' metadata={'id':
179.0}
```

Figure 6.20 – Output example from the get_relevant_documents function